DATE:15/10/2026

ASSESSMENT TOOL-3 CODING CHALLENGE

APPLICATION BASED QUESTIONS

COURSE CODE : DSA0308

COURSE NAME: Natural Language Processing

Reg No: 192472008 **Name: C.Manoj Kumar**

---

1. Email, Password, and Mobile Number Validation Using Regular Expressions

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

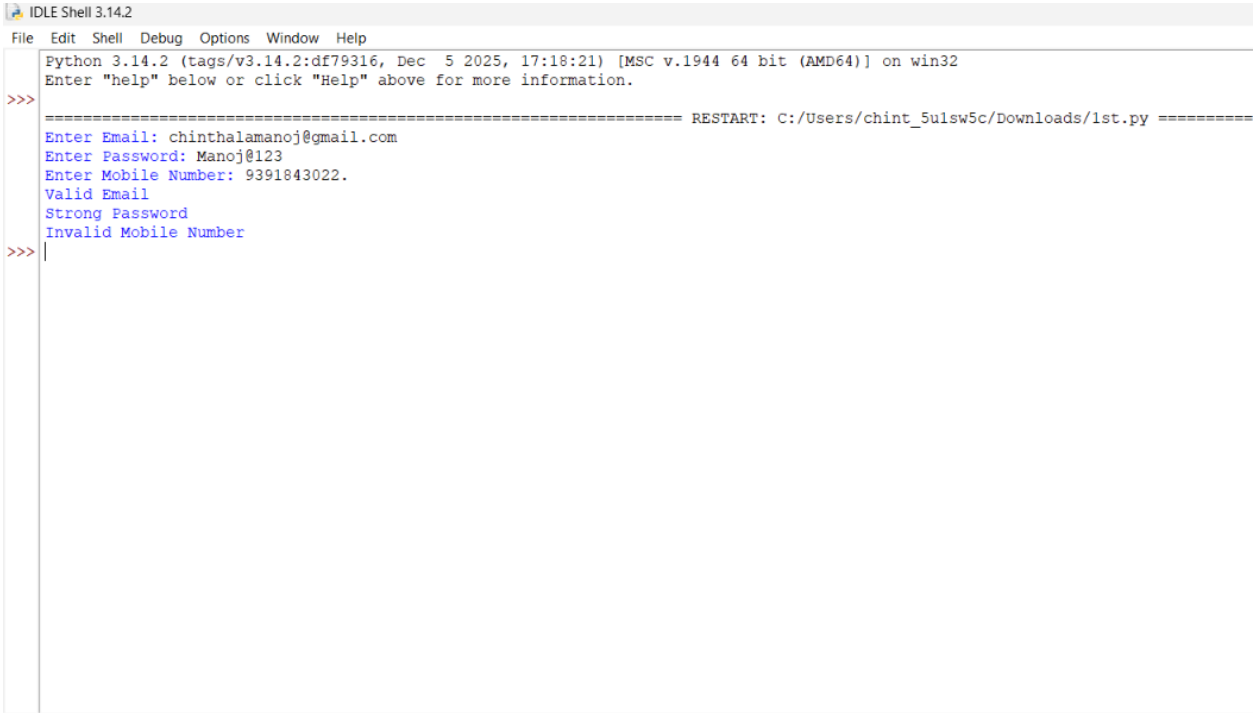
if re.fullmatch(r'[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)', email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(r'(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}', password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(r'[6-9]\d{9}', mobile):
    print("Valid Mobile Number")
```

```
else:
    print("Invalid Mobile Number")
```

OUT PUT:



2. DFA (Deterministic Finite Automaton) Simulator

```
def check_string(string):
    state = "q0"
    path = [state]

    for ch in string:
        if ch == 'a':
            if state == "q0":
                state = "q1"
            elif state == "q1":
                state = "q1"
            else:
                state = "q1"

        elif ch == 'b':
            if state == "q0":
                state = "q0"
            elif state == "q1":
                state = "q2"
            else:
                state = "q0"

        else:
            print("Invalid input")
            return

        path.append(state)

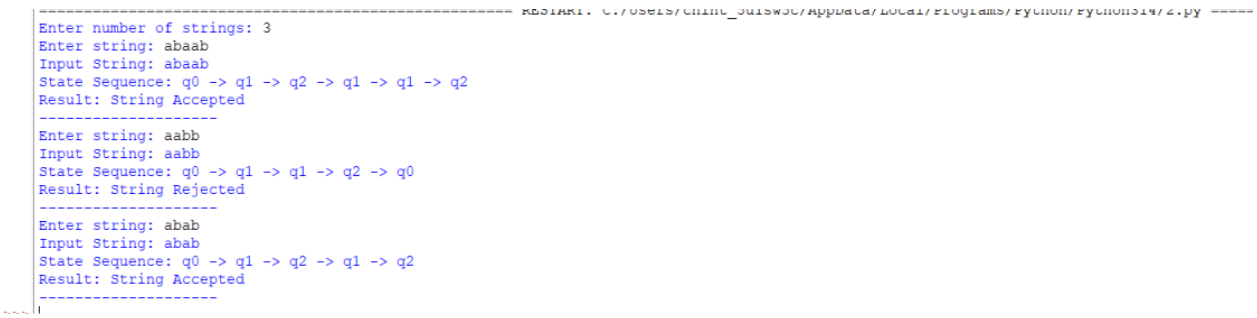
    print("Input String:", string)
    print("State Sequence:", " -> ".join(path))
```

```
    if state == "q2":
        print("Result: String Accepted")
    else:
        print("Result: String Rejected")


n = int(input("Enter number of strings: "))

for i in range(n):
    s = input("Enter string: ")
    check_string(s)
    print("--------------------")
```

OUT PUT:



3. Smart Pattern Matching Engine Using Regular Expressions

```
import re

def search_pattern(text, choice):
    if choice == "1":
        term = input("Enter word: ").strip()
        if not term:
            print("Search term cannot be empty")
            return
        pattern = r'\b' + re.escape(term) + r'\b'

    elif choice == "2":
        pattern = r'\b(?:0[1-9]|[12]\d|3[01])/(?:0[1-9]|1[0-2])/\d{4}\b'

    elif choice == "3":
        pattern = r'\b[6-9]\d{9}\b'

    elif choice == "4":
        pattern = r'(?<!\w)#\w+'

    elif choice == "5":
        pattern = r'(?<!\w)@\w+'

    elif choice == "6":
        term = input("Enter prefix: ").strip()
        if not term:
            print("Prefix cannot be empty")
            return
        pattern = r'\b' + re.escape(term) + r'\w*'
```

```
    elif choice == "7":
        term = input("Enter suffix: ").strip()
        if not term:
            print("Suffix cannot be empty")
            return
        pattern = r'\w*' + re.escape(term) + r'\b'

    else:
        print("Invalid choice")
        return

    matches = re.findall(pattern, text, re.IGNORECASE)

    if matches:
        print("Matching patterns:")
        for item in matches:
            print(item)
        print("Total matches:", len(matches))
    else:
        print("No matches found")



def main():
    text = input("Enter text: ")

    if not text.strip():
        print("Text cannot be empty")
        return

    while True:
        print("\n--- SMART PATTERN MATCHING ENGINE ---")
        print("1. Search Word")
```

```
        print("2. Search Date")
        print("3. Search Phone Number")
        print("4. Search Hashtag")
        print("5. Search Mention")
        print("6. Search Prefix")
        print("7. Search Suffix")
        print("8. Exit")

        choice = input("Enter choice: ").strip()

        if choice == "8":
            print("Program ended")
            break

        search_pattern(text, choice)


if __name__ == "__main__":
    main()
```

OUT PUT:

```
8. Exit
Enter choice: 3
Matching patterns:
9876543210
Total matches: 1

--- SMART PATTERN MATCHING ENGINE ---
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
8. Exit
Enter choice: 4
Matching patterns:
#NLP
Total matches: 1

--- SMART PATTERN MATCHING ENGINE ---
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
8. Exit
Enter choice: 2
Matching patterns:
12/09/2026
Total matches: 1

--- SMART PATTERN MATCHING ENGINE ---
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
8. Exit
Enter choice:
```